

Documentación Técnica — Sistema de Gestión Hockey Pista

Índice

1. Descripción general
2. Stack tecnológico
3. Arquitectura del sistema
4. Estructura del repositorio
5. Configuración del entorno
6. Levantar el proyecto
7. Base de datos
8. Backend — API REST
9. Frontend — React
10. Autenticación y seguridad
11. Migraciones
12. Backups
13. Deploy en producción

Descripción general

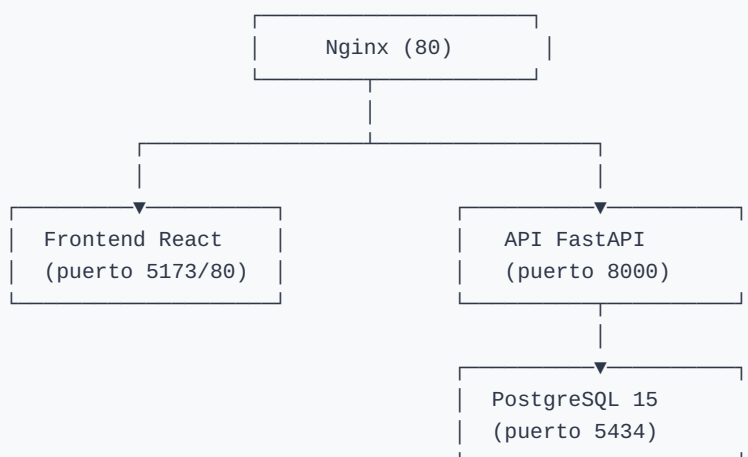
Sistema web de gestión de torneos de hockey sobre pista. Incluye:

- Gestión de clubes, equipos, personas y planteles
- Carga de planillas de partidos (goles, tarjetas, participantes)
- Fixture y tabla de posiciones automática
- Estadísticas: goleadores, valla menos vencida, tarjetas acumuladas
- Sección de noticias
- Sistema de usuarios con roles y permisos
- Auditoría completa de cambios
- Sitio público sin autenticación + panel administrativo

Stack tecnológico

Capa	Tecnología	Versión
Backend	Python + FastAPI	3.x / 0.104.1
ORM	SQLAlchemy	2.0.23
Validación	Pydantic v2	2.5.0
Migraciones	Alembic	1.12.1
Base de datos	PostgreSQL	15
Frontend	React + TypeScript	19.x / ~5.9
Bundler	Vite	7.x
Router	React Router DOM	7.x
HTTP client	Axios	1.x
PDF	jsPDF + autotable	2.5 / 3.8
Proxy	Nginx	alpine
Contenedores	Docker + Compose	-
Autenticación	JWT (python-jose) + bcrypt/argon2	-
Email	Resend	-
Rate limiting	SlowAPI	0.1.9

Arquitectura del sistema



Los cuatro servicios corren en la red interna `hockey-network` vía Docker Compose. Solo Nginx expone puertos al exterior. La base de datos solo acepta conexiones desde `127.0.0.1:5434` en el host (no expuesta públicamente).

Flujo de una request:

1. El navegador hace una request a `tu-dominio.com.ar`
2. Nginx enruta: `/api/*` → FastAPI, todo lo demás → React
3. React consulta la API bajo `/api/`
4. FastAPI valida JWT, aplica permisos por rol y responde

Estructura del repositorio

```
/
├── backend/
│   ├── app/
│   │   ├── auth/           # Autenticación: login, JWT, invitaciones, recuperación
│   │   ├── core/           # Config, excepciones, email, middleware, auditoría
│   │   ├── dependencies/    # Dependencias FastAPI: auth y permisos
│   │   ├── models/         # Modelos SQLAlchemy (una tabla = un archivo)
│   │   ├── routers/        # Endpoints REST (uno por entidad)
│   │   ├── schemas/        # Schemas Pydantic (request/response)
│   │   ├── services/       # Lógica de negocio (separada de los routers)
│   │   ├── utils/          # Helpers (db session, etc.)
│   │   ├── database.py     # Configuración de la sesión SQLAlchemy
│   │   └── main.py         # Punto de entrada FastAPI
│   ├── db/
│   │   ├── init/           # Scripts SQL de inicialización (ejecutan en orden)
│   │   └── seed/           # Datos de prueba para desarrollo
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── api/            # Funciones de llamada a la API (por entidad)
│   │   ├── pages/
│   │   │   ├── public/     # Páginas sin autenticación
│   │   │   └── admin/      # Páginas del panel administrativo
│   │   └── components/     # Componentes reutilizables
│   └── package.json
├── backups/                # Scripts de backup y restauración
├── docs/                   # Documentación
├── nginx.conf              # Config Nginx producción
├── nginx.local.conf        # Config Nginx desarrollo local
├── docker-compose.yml      # Compose base
└── docker-compose.override.yml # Overrides para desarrollo
```

Configuración del entorno

El proyecto usa un archivo `.env` en la raíz. Crear a partir del siguiente template:

```
# Base de datos
POSTGRES_DB=hockey
POSTGRES_USER=hockey_user
POSTGRES_PASSWORD=<contraseña-segura>
DATABASE_URL=postgresql://hockey_user:<contraseña>@db:5432/hockey

# JWT
JWT_SECRET=<string-aleatorio-largo>
JWT_ALGORITHM=HS256

# Tokens
ACCESS_TOKEN_EXPIRE_MINUTES=1440
REFRESH_TOKEN_EXPIRE_DAYS=30

# Email (Resend)
RESEND_API_KEY=<api-key-de-resend>
FRONTEND_URL=https://tu-dominio.com.ar

# Cookies
COOKIE_SECURE=True           # False en desarrollo local
COOKIE_SAMESITE=strict

# App
ENVIRONMENT=production       # o "development"
ENABLE_SWAGGER=False         # True para habilitar /api/docs

# Frontend
VITE_API_URL=https://tu-dominio.com.ar
FRONTEND_TARGET=production   # o "development"

# Nginx
HOST_PORT=80

# CORS
CORS_ALLOW_ORIGINS=https://tu-dominio.com.ar
```

Importante: nunca subir el `.env` al repositorio. Está en `.gitignore`.

Levantar el proyecto

Requisitos previos

- Docker y Docker Compose instalados
- Archivo `.env` configurado en la raíz

Desarrollo local

En desarrollo, Docker Compose carga automáticamente `docker-compose.yml` y `docker-compose.override.yml`. El override aplica las siguientes diferencias respecto a producción:

Servicio	Cambio en desarrollo
nginx	Usa <code>nginx.local.conf</code> en lugar de <code>nginx.conf</code>
frontend	Expone el puerto <code>5173</code> al host
api	Expone el puerto <code>8000</code> al host y monta la carpeta <code>tests/</code>
api	Ejecuta <code>entrypoint.sh</code> en lugar de uvicorn directo

```
# Primera vez (construye imágenes y levanta)
docker compose up --build

# Veces siguientes
docker compose up

# Detener
docker compose down

# Detener y eliminar volúmenes (limpia la base de datos)
docker compose down -v
```

El frontend queda disponible en `http://localhost:5173` y la API en `http://localhost:8000/api/`.

En producción el override **no existe**, así que Docker solo usa `docker-compose.yml`. Los puertos de la API y el frontend no se exponen al exterior; todo el tráfico pasa por Nginx.

Con Swagger habilitado (desarrollo)

Agregar en `.env`:

```
ENABLE_SWAGGER=True
```

Luego acceder a `http://localhost:8000/api/docs`.

Seed de datos de prueba

Los scripts en `backend/db/seed/` se ejecutan automáticamente al iniciar el contenedor si la base de datos está vacía. Incluyen clubes, equipos, personas, torneos, usuarios y más.

Base de datos

Inicialización

Los scripts en `backend/db/init/` se ejecutan en orden al crear el contenedor por primera vez:

Archivo	Contenido
<code>001_enums.sql</code>	Tipos enumerados (roles, categorías, géneros, estados)
<code>002_tables.sql</code>	Tablas base: club, equipo, persona, fichaje, planteles, torneos, posiciones
<code>003_tables_extra.sql</code>	Tablas adicionales: usuario, refresh_token, fixture, noticias
<code>004_functions.sql</code>	Funciones PL/pgSQL auxiliares
<code>005_triggers.sql</code>	Triggers (ej: actualización automática de timestamps)
<code>006_views.sql</code>	Vistas: posiciones, estadísticas
<code>007_grants.sql</code>	Permisos por usuario de base de datos
<code>008_auditoria.sql</code>	Tabla y triggers de auditoría
<code>009_index.sql</code>	Índices para performance

Convenciones de tablas

- **Soft delete:** las entidades principales tienen `borrado_en TIMESTAMP`. Un registro con `borrado_en IS NOT NULL` está eliminado lógicamente pero se conserva en la base.
- **Auditoría:** todos los modelos tienen `creado_en`, `actualizado_en`, `creado_por`, `actualizado_por`.
- **IDs:** `INT GENERATED ALWAYS AS IDENTITY` (autoincremental nativo de PostgreSQL).

Tipos enumerados (ENUMs)

Enum	Valores
tipo_genero	MASCULINO, FEMENINO, MIXTO
tipo_categoria	MAYORES, SUB_19, SUB_16, SUB_14, SUB_12
tipo_rol_persona	JUGADOR, DT, ARBITRO, ASISTENTE, MEDICO, PREPARADOR_FISICO, DELEGADO
tipo_estado_partido	BORRADOR, PENDIENTE, TERMINADO, SUSPENDIDO, ANULADO, REPROGRAMADO
tipo_tarjeta	VERDE, AMARILLA, ROJA
tipo_estado_tarjeta	VALIDA, ANULADA, CORREGIDA
tipo_gol	GJ (jugada), GC (córner), GP (penal), DP (desvío penal)
tipo_estado_gol	VALIDO, ANULADO, CORREGIDO
tipo_suspension	POR_PARTIDOS, POR_FECHA
tipo_estado_suspension	ACTIVA, CUMPLIDA, ANULADA
tipo_fase	LIGA, ELIMINACION, GRUPOS
tipo_torneo	LIGA, PLAYOFF, COPA
tipo_usuario	SUPERUSUARIO, ADMIN, EDITOR, LECTOR

Entidades y relaciones

Diagrama general



Descripción de cada tabla

club

Organización deportiva. Raíz de la jerarquía.

Columna	Tipo	Descripción
<code>id_club</code>	INT PK	Identificador
<code>nombre</code>	VARCHAR(100)	Nombre del club
<code>provincia</code>	VARCHAR(100)	Provincia
<code>ciudad</code>	VARCHAR(100)	Ciudad
<code>direccion</code>	VARCHAR(200)	Dirección (opcional)
<code>telefono</code>	VARCHAR(20)	Teléfono (opcional)
<code>email</code>	VARCHAR(100)	Email (opcional)

Restricción: `(nombre, ciudad)` únicos.

`equipo`

Equipo competitivo de un club en una categoría y género específicos.

Columna	Tipo	Descripción
<code>id_equipo</code>	INT PK	Identificador
<code>nombre</code>	VARCHAR(100)	Nombre del equipo
<code>id_club</code>	INT FK → club	Club al que pertenece
<code>categoria</code>	tipo_categoria	Categoría (MAYORES, SUB_19, etc.)
<code>division</code>	VARCHAR(30)	División (opcional)
<code>genero</code>	tipo_genero	Género del equipo

Restricción: `(id_club, nombre, categoria, division, genero)` únicos.

`persona`

Individuo del sistema (jugador, árbitro, DT, delegado, etc.).

Columna	Tipo	Descripción
<code>id_persona</code>	INT PK	Identificador
<code>documento</code>	INT UNIQUE	DNI (opcional, único si presente)
<code>nombre</code>	VARCHAR(100)	Nombre
<code>apellido</code>	VARCHAR(100)	Apellido
<code>fecha_nacimiento</code>	DATE	Fecha de nacimiento (opcional)
<code>genero</code>	tipo_genero	Género
<code>telefono</code>	VARCHAR(20)	Teléfono (opcional)
<code>email</code>	VARCHAR(100)	Email (opcional)

`persona_rol`

Roles que tiene una persona (puede tener múltiples a lo largo del tiempo).

Columna	Tipo	Descripción
<code>id_persona_rol</code>	INT PK	Identificador
<code>id_persona</code>	INT FK → persona	Persona
<code>rol</code>	tipo_rol_persona	Rol (JUGADOR, ARBITRO, etc.)
<code>fecha_desde</code>	DATE	Inicio del rol
<code>fecha_hasta</code>	DATE	Fin del rol (NULL = activo)

`fichaje_rol`

Vinculación de una persona (en un rol específico) con un club por un período.

Columna	Tipo	Descripción
<code>id_fichaje_rol</code>	INT PK	Identificador
<code>id_persona</code>	INT FK → persona	Persona fichada
<code>id_club</code>	INT FK → club	Club al que se ficha
<code>id_persona_rol</code>	INT FK → persona_rol	Rol habilitante
<code>rol</code>	tipo_rol_persona	Rol en este fichaje
<code>fecha_inicio</code>	DATE	Inicio del fichaje
<code>fecha_fin</code>	DATE	Fin del fichaje (NULL = activo)
<code>activo</code>	BOOLEAN	Estado del fichaje

Restricciones: una persona no puede tener el mismo rol activo en dos clubes distintos, ni duplicar el mismo rol activo en el mismo club.

plantel

Grupo de integrantes de un equipo en una temporada. Solo puede haber un plantel activo por equipo a la vez.

Columna	Tipo	Descripción
id_plantel	INT PK	Identificador
id_equipo	INT FK → equipo	Equipo
nombre	VARCHAR(100)	Nombre del plantel
temporada	VARCHAR(10)	Ej: 2024 o 2024-2025
fecha_apertura	DATE	Fecha de apertura
fecha_cierre	DATE	Fecha de cierre (NULL = abierto)
activo	BOOLEAN	Estado del plantel

Restricción: solo un plantel activo por equipo (índice parcial).

plantel_integrante

Vincula a una persona con un plantel en un rol y número de camiseta.

Columna	Tipo	Descripción
id_plantel_integrante	INT PK	Identificador
id_plantel	INT FK → plantel	Plantel
id_persona	INT FK → persona	Persona
id_fichaje_rol	INT FK → fichaje_rol	Fichaje que habilita (opcional)
rol_en_plantel	tipo_rol_persona	Rol dentro del plantel
numero_camiseta	INT	Número de camiseta (opcional)
fecha_alta	DATE	Ingreso al plantel
fecha_baja	DATE	Baja del plantel (NULL = activo)

Restricción: (id_plantel, id_persona) únicos.

torneo

Competencia oficial en la que participan equipos.

Columna	Tipo	Descripción
<code>id_torneo</code>	INT PK	Identificador
<code>nombre</code>	VARCHAR(100)	Nombre del torneo
<code>categoria</code>	tipo_categoria	Categoría
<code>division</code>	VARCHAR(30)	División (opcional)
<code>genero</code>	tipo_genero	Género
<code>tipo</code>	tipo_torneo	Tipo de competencia: <code>LIGA</code> , <code>PLAYOFF</code> , <code>COPA</code> (default: <code>LIGA</code>)
<code>fecha_inicio</code>	DATE	Inicio del torneo
<code>fecha_fin</code>	DATE	Fin del torneo (opcional)
<code>activo</code>	BOOLEAN	Si el torneo está en curso

`inscripcion_torneo`

Inscripción de un equipo en un torneo. Es la tabla de unión entre ambos y se usa como referencia en los partidos (en lugar de referenciar directamente al equipo).

Columna	Tipo	Descripción
<code>id_inscripcion</code>	INT PK	Identificador
<code>id_equipo</code>	INT FK → equipo	Equipo inscripto
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>fecha_inscripcion</code>	DATE	Fecha de inscripción
<code>fecha_baja</code>	TIMESTAMP	Baja de la inscripción (NULL = activa)

Restricción: (`id_equipo`, `id_torneo`) únicos.

`fase`

Etapas dentro de un torneo (liga, grupos, eliminación).

Columna	Tipo	Descripción
<code>id_fase</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>nombre</code>	VARCHAR(50)	Nombre de la fase
<code>tipo</code>	tipo_fase	Tipo (LIGA, GRUPOS, ELIMINACION)
<code>orden</code>	INT	Orden de la fase en el torneo
<code>fecha_inicio</code>	DATE	Inicio (opcional)
<code>fecha_fin</code>	DATE	Fin (opcional)

`partido`

Registro de un partido jugado entre dos equipos inscriptos en un torneo.

Columna	Tipo	Descripción
<code>id_partido</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>id_fase</code>	INT FK → fase	Fase (opcional)
<code>fecha</code>	DATE	Fecha del partido
<code>horario</code>	TIME	Horario (opcional)
<code>id_inscripcion_local</code>	INT FK → inscripcion_torneo	Equipo local
<code>id_inscripcion_visitante</code>	INT FK → inscripcion_torneo	Equipo visitante
<code>id_arbitro1</code>	INT FK → persona	Árbitro principal (opcional)
<code>id_arbitro2</code>	INT FK → persona	Árbitro secundario (opcional)
<code>id_capitan_local</code>	INT FK → plantel_integrante	Capitán local (opcional)
<code>id_capitan_visitante</code>	INT FK → plantel_integrante	Capitán visitante (opcional)
<code>ubicacion</code>	VARCHAR(200)	Cancha (opcional)
<code>numero_fecha</code>	INT	Número de jornada (opcional)
<code>estado_partido</code>	tipo_estado_partido	Estado del partido
<code>goles_local_manual</code>	INT	Resultado manual (para categorías sin desglose)
<code>goles_visitante_manual</code>	INT	Resultado manual

Restricción: no puede haber dos partidos entre los mismos equipos en la misma fecha y torneo (índice único sobre `LEAST` / `GREATEST` de inscripciones).

`participan_partido`

Jugadores y participantes que estuvieron en un partido.

Columna	Tipo	Descripción
<code>id_participante_partido</code>	INT PK	Identificador
<code>id_partido</code>	INT FK → partido	Partido
<code>id_plantel_integrante</code>	INT FK → plantel_integrante	Integrante del plantel
<code>numero_camiseta</code>	INT	Número de camiseta usado en este partido

Restricción: `(id_partido, id_plantel_integrante)` únicos.

`gol`

Gol registrado en un partido, ligado al participante que lo convirtió.

Columna	Tipo	Descripción
<code>id_gol</code>	INT PK	Identificador
<code>id_partido</code>	INT FK → partido	Partido
<code>id_participante_partido</code>	INT FK → participan_partido	Jugador que convirtió
<code>minuto</code>	INT	Minuto (opcional)
<code>cuarto</code>	INT (1-4)	Cuarto de juego (opcional)
<code>referencia_gol</code>	tipo_gol	Tipo: GJ, GC, GP, DP
<code>es_autogol</code>	BOOLEAN	Si fue autogol
<code>estado_gol</code>	tipo_estado_gol	VALIDO, ANULADO, CORREGIDO
<code>motivo_anulacion</code>	VARCHAR(500)	Motivo si fue anulado

`tarjeta`

Tarjeta disciplinaria recibida por un participante en un partido.

Columna	Tipo	Descripción
<code>id_tarjeta</code>	INT PK	Identificador
<code>id_partido</code>	INT FK → partido	Partido
<code>id_participante_partido</code>	INT FK → participan_partido	Jugador sancionado
<code>tipo</code>	tipo_tarjeta	VERDE, AMARILLA, ROJA
<code>minuto</code>	INT	Minuto (opcional)
<code>cuarto</code>	INT (1-4)	Cuarto (opcional)
<code>estado_tarjeta</code>	tipo_estado_tarjeta	VALIDA, ANULADA, CORREGIDA
<code>revisada</code>	BOOLEAN	Si fue revisada/apelada

suspension

Sanción que impide a una persona participar en partidos futuros.

Columna	Tipo	Descripción
<code>id_suspension</code>	INT PK	Identificador
<code>id_persona_rol</code>	INT FK → persona_rol	Persona y rol sancionado
<code>id_torneo</code>	INT FK → torneo	Torneo donde aplica
<code>id_partido_origen</code>	INT FK → partido	Partido que originó la sanción
<code>tipo_suspension</code>	tipo_suspension	POR_PARTIDOS o POR_FECHA
<code>motivo</code>	VARCHAR(500)	Motivo de la sanción
<code>fechas_suspension</code>	INT	Cantidad de partidos (si es POR_PARTIDOS)
<code>fecha_fin_suspension</code>	DATE	Fecha límite (si es POR_FECHA)
<code>cumplidas</code>	INT	Partidos cumplidos
<code>partidos_cumplidos</code>	INT[]	Array con IDs de partidos cumplidos
<code>estado_suspension</code>	tipo_estado_suspension	ACTIVA, CUMPLIDA, ANULADA

posicion

Estadísticas de un equipo en un torneo (tabla de posiciones). Se actualiza al cargar planillas.

Columna	Tipo	Descripción
<code>id_posicion</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>id_equipo</code>	INT FK → equipo	Equipo
<code>puntos</code>	INT	Puntos acumulados
<code>partidos_jugados</code>	INT	PJ
<code>ganados</code>	INT	Ganados
<code>empatados</code>	INT	Empatados
<code>perdidos</code>	INT	Perdidos
<code>goles_a_favor</code>	INT	GF
<code>goles_en_contra</code>	INT	GC
<code>diferencia_gol</code>	INT (generado)	GF - GC (columna calculada en la BD)

`fixture_fecha`

Agrupación lógica de jornadas dentro de un torneo. Almacena el número de fecha y, si hay número impar de equipos, qué equipo descansa en esa jornada.

Columna	Tipo	Descripción
<code>id_fixture_fecha</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>numero_fecha</code>	INT	Número de la jornada
<code>rueda</code>	VARCHAR(20)	Rueda: <code>ida</code> o <code>vuelta</code>
<code>id_equipo_descansa</code>	INT FK → equipo	Equipo que descansa (NULL si número par de equipos)
<code>creado_en</code>	TIMESTAMP	Fecha de creación
<code>creado_por</code>	VARCHAR(100)	Usuario que creó
<code>actualizado_en</code>	TIMESTAMP	Última actualización
<code>actualizado_por</code>	VARCHAR(100)	Usuario que actualizó

`fixture_playoff_ronda`

Ronda de un bracket de playoff (Final, Semifinal, Cuartos, etc.). Agrupa los partidos de cada eliminatoria.

Columna	Tipo	Descripción
<code>id_fixture_playoff_ronda</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>nombre</code>	VARCHAR(100)	Nombre de la ronda (ej: "Semifinal", "Final")
<code>orden</code>	INT	Orden dentro del bracket (1 = primera ronda)
<code>ida_y_vuelta</code>	BOOLEAN	Si la ronda se juega a ida y vuelta
<code>creado_en</code>	TIMESTAMP	Fecha de creación
<code>creado_por</code>	VARCHAR(100)	Usuario que creó

Restricción: `(id_torneo, orden)` únicos.

`fixture_partido`

Partido programado (fixture) antes de jugarse. Al cargarse la planilla real queda vinculado a `partido`. Para partidos de playoff con equipos aún no definidos, `id_equipo_local` / `id_equipo_visitante` son NULL y se usan los campos `placeholder_*`.

Columna	Tipo	Descripción
<code>id_fixture_partido</code>	INT PK	Identificador
<code>id_torneo</code>	INT FK → torneo	Torneo
<code>id_equipo_local</code>	INT FK → equipo (nullable)	Equipo local (NULL en playoff hasta definirse)
<code>id_equipo_visitante</code>	INT FK → equipo (nullable)	Equipo visitante (NULL en playoff hasta definirse)
<code>id_fixture_fecha</code>	INT FK → fixture_fecha	Jornada a la que pertenece (NULL en playoff)
<code>id_fixture_playoff_ronda</code>	INT FK → fixture_playoff_ronda	Ronda de playoff (NULL en fixture de liga)
<code>placeholder_local</code>	VARCHAR(100)	Ej: "Ganador SF1" mientras no hay equipo definido
<code>placeholder_visitante</code>	VARCHAR(100)	Ej: "Ganador SF2"
<code>fecha_programada</code>	DATE	Fecha programada (opcional, asignada manualmente)
<code>horario</code>	TIME	Horario (opcional)
<code>ubicacion</code>	VARCHAR(200)	Lugar (opcional)
<code>numero_fecha</code>	INT	Jornada (copia desnormalizada)
<code>estado</code>	tipo_estado_partido	Estado del partido de fixture
<code>id_partido_real</code>	INT FK → partido	Partido real (NULL hasta que se cargue la planilla)
<code>creado_en</code>	TIMESTAMP	Fecha de creación
<code>creado_por</code>	VARCHAR(100)	Usuario que creó
<code>actualizado_por</code>	VARCHAR(100)	Usuario que actualizó

Estados de `fixture_partido`:

Estado	Visible al público	Descripción
<code>BORRADOR</code>	No	Creado automáticamente por el generador. Sin fecha asignada.
<code>PENDIENTE</code>	Sí	Tiene fecha asignada. Se muestra en el fixture público.
<code>TERMINADO</code>	Sí	Partido jugado y planilla cargada.
<code>SUSPENDIDO</code>	Sí	Partido suspendido.
<code>REPROGRAMADO</code>	Sí	Partido reprogramado.
<code>ANULADO</code>	No (en fixture)	Partido anulado.

Transición automática: cuando se asigna una `fecha_programada` a un partido en estado `BORRADOR`, el backend lo pasa automáticamente a `PENDIENTE`. Si se quita la fecha de un `PENDIENTE`, vuelve a `BORRADOR`.

usuario

Usuario del sistema con acceso al panel administrativo.

Columna	Tipo	Descripción
<code>id_usuario</code>	INT PK	Identificador
<code>username</code>	VARCHAR(50) UNIQUE	Nombre de usuario
<code>email</code>	VARCHAR(100) UNIQUE	Email
<code>password_hash</code>	TEXT	Contraseña hasheada (argon2)
<code>tipo</code>	tipo_usuario	Rol: SUPERUSUARIO, ADMIN, EDITOR, LECTOR
<code>activo</code>	BOOLEAN	Si puede iniciar sesión
<code>intentos_fallidos</code>	INT	Intentos de login fallidos
<code>bloqueado_hasta</code>	TIMESTAMP	Bloqueo temporal por intentos
<code>ultimo_login</code>	TIMESTAMP	Último acceso

refresh_token

Tokens de sesión de larga duración para renovar el access token sin re-login.

Columna	Tipo	Descripción
<code>id_refresh_token</code>	INT PK	Identificador
<code>id_usuario</code>	INT FK → usuario	Usuario
<code>token_hash</code>	TEXT	Hash del token (no se guarda el token en texto plano)
<code>expires_at</code>	TIMESTAMP	Expiración
<code>revoked</code>	BOOLEAN	Si fue revocado
<code>created_by_ip</code>	INET	IP de creación
<code>user_agent</code>	TEXT	User-agent del cliente

noticias

Entradas de contenido publicables en el sitio.

Columna	Tipo	Descripción
<code>id_noticia</code>	INT PK	Identificador
<code>titulo</code>	VARCHAR(255)	Título
<code>imagen_url</code>	TEXT	URL de imagen (opcional)
<code>epigrafe</code>	VARCHAR(255)	Epígrafe/bajada (opcional)
<code>texto</code>	TEXT	Cuerpo de la noticia
<code>url_externa</code>	TEXT	URL de referencia externa (opcional)

Vistas SQL

La tabla de posiciones se consulta desde una vista SQL (`006_views.sql`) que agrega los resultados de partidos en estado `TERMINADO`. No requiere actualización manual; se recalcula en cada consulta.

Backend — API REST

Estructura de un módulo

Cada entidad sigue el patrón:

```
models/entidad.py    → Modelo SQLAlchemy (mapeo a tabla)
schemas/entidad.py   → Schemas Pydantic (validación entrada/salida)
services/entidad.py  → Lógica de negocio (queries, reglas)
routers/entidad.py   → Endpoints FastAPI (HTTP handlers)
```

Prefijo base

Todos los endpoints tienen el prefijo `/api`. Ejemplo: `GET /api/clubes`.

Endpoints disponibles

Auth (/api/auth)

Método	Ruta	Descripción	Rol mínimo
POST	/login	Iniciar sesión	Público
POST	/refresh	Renovar access token	Autenticado
POST	/logout	Cerrar sesión	Autenticado
GET	/me	Datos del usuario actual	Autenticado
POST	/invitar	Invitar nuevo usuario por email	Superusuario
POST	/recuperar-password	Enviar email de recuperación	Público
POST	/reset-password-confirm	Confirmar nueva contraseña	Público
PATCH	/cambiar-password	Cambiar contraseña propia	Autenticado
GET	/usuarios	Listar usuarios	Superusuario
PATCH	/usuarios/{id}/rol	Cambiar rol de usuario	Superusuario
PATCH	/usuarios/{id}/estado	Activar/desactivar usuario	Superusuario

Clubes (/api/clubes)

Método	Ruta	Rol mínimo
GET	/	Público
GET	/id	Público
POST	/	Superusuario
PUT	/id	Superusuario
DELETE	/id	Superusuario
POST	/id/restore	Superusuario

Equipos (/api/equipos)

Método	Ruta	Rol mínimo
GET	/	Público
GET	/ {id}	Público
POST	/	Admin
PUT	/ {id}	Admin
DELETE	/ {id}	Superusuario
POST	/ {id}/restore	Superusuario

Personas (/api/personas)

Método	Ruta	Rol mínimo
GET	/	Editor
GET	/ {id}	Editor
POST	/	Admin
PUT	/ {id}	Admin
DELETE	/ {id}	Admin
POST	/ {id}/roles	Admin
PUT	/ {id}/roles/{rol_id}	Admin

Torneos (/api/torneos)

Método	Ruta	Rol mínimo
GET	/	Público
GET	/ {id}	Público
POST	/	Superusuario
PUT	/ {id}	Superusuario
DELETE	/ {id}	Superusuario
POST	/ {id}/finalizar	Superusuario
POST	/ {id}/reabrir	Superusuario
POST	/ {id}/restaurar	Superusuario

Inscripciones (`/api/torneos/{id}/inscripciones`)

Método	Ruta	Rol mínimo
GET	<code>/</code>	Público
POST	<code>/</code>	Admin
DELETE	<code>/ {id_equipo} /BAJA</code>	Admin

Partidos (`/api/partidos`)

Método	Ruta	Rol mínimo
GET	<code>/recientes</code>	Público
GET	<code>/ {id}</code>	Público
GET	<code>/equipos/ {id_equipo}</code>	Público
POST	<code>/planilla</code>	Editor
PUT	<code>/planilla/ {id}</code>	Editor
DELETE	<code>/ {id}</code>	Superusuario

Fixture (`/api/fixture`)

Método	Ruta	Descripción	Rol mínimo
GET	<code>/proximos</code>	Próximos partidos públicos (PENDIENTE, SUSPENDIDO, REPROGRAMADO)	Público
GET	<code>/torneo/{id}</code>	Fixture de un torneo (solo estados visibles)	Público
GET	<code>/admin/torneo/{id}</code>	Fixture de un torneo (todos los estados, incluye BORRADOR)	Editor
GET	<code>/ {id}</code>	Detalle de un partido de fixture	Público
POST	<code>/</code>	Programar partido manual	Admin
POST	<code>/preview/{id_torneo}</code>	Previsualizar fixture automático (sin guardar)	Admin
POST	<code>/generar/{id_torneo}</code>	Generar y guardar fixture automático (round-robin)	Admin
PUT	<code>/ {id}</code>	Editar partido de fixture	Admin
DELETE	<code>/ {id}</code>	Eliminar un partido de fixture	Admin
DELETE	<code>/torneo/{id_torneo}</code>	Eliminar todo el fixture de un torneo	Admin
GET	<code>/playoff/rondas/{id_torneo}</code>	Listar rondas de un playoff	Editor
POST	<code>/playoff/rondas/{id_torneo}</code>	Crear ronda de playoff manualmente	Admin
POST	<code>/playoff/preview/{id_torneo}</code>	Previsualizar bracket de playoff (sin guardar)	Admin
POST	<code>/playoff/generar/{id_torneo}</code>	Generar y guardar bracket de playoff	Admin

Importante: el orden de las rutas en FastAPI es relevante. Las rutas literales (`/proximos`, `/admin/torneo/{id}`, `/torneo/{id}`, `/preview/{id}`, `/generar/{id}`) deben registrarse **antes** de la ruta paramétrica `/ {id}` para evitar que FastAPI trate cadenas como "torneo" como un ID entero.

Tipos de fixture (`tipo`):

Valor	Descripción
<code>simple</code>	Todos contra todos una sola vez (round-robin ida)
<code>ida_y_vuelta</code>	Vuelta espejo exacto de la ida, mismos enfrentamientos con local/visitante invertidos
<code>ida_y_vuelta_aleatorio</code>	Vuelta con orden de fechas aleatorio; se garantiza que la primera fecha de vuelta no repita los enfrentamientos de la última de ida

El campo `rueda` (`"ida"` o `"vuelta"`) se almacena en `fixture_fecha` y se expone en el response de cada `fixture_partido`. Para torneos sin generación automática (programados manualmente) el campo es `null`.

Generación automática de fixture (round-robin):

1. El frontend llama a `POST /preview/{id_torneo}` con `{ tipo: "simple" | "ida_y_vuelta" | "ida_y_vuelta_aleatorio" }`.
2. El backend devuelve `FixturePreviewResponse` con todos los partidos y descansos por fecha, sin guardar nada.
3. El usuario revisa el preview y confirma.
4. El frontend llama a `POST /generar/{id_torneo}` con el mismo tipo.
5. El backend guarda `fixture_fecha` y `fixture_partido` en estado `BORRADOR`.
6. Si ya existía fixture para ese torneo, se elimina primero (incluyendo partidos jugados si los hay — el sistema avisa al frontend).

Generación de bracket de playoff:

Disponible para torneos de tipo `PLAYOFF` o `COPA`. Genera un cuadro de eliminación directa a partir de los equipos inscriptos.

Parámetros (`GenerarPlayoffRequest`):

Campo	Tipo	Descripción
<code>formato</code>	<code>"ida"</code> <code>"ida_y_vuelta"</code>	Si cada serie se juega a ida o ida y vuelta
<code>asignacion</code>	<code>"automatico"</code> <code>"manual"</code>	Cómo se asignan los cruces
<code>duelos</code>	lista (opcional)	Pares <code>{id_equipo_local, id_equipo_visitante}</code> para la primera ronda (solo para <code>asignacion: "manual"</code>)

Algoritmo:

- Se calculan las rondas necesarias según la cantidad de equipos (potencia de 2 superior).
- Si el número de equipos no es potencia de 2, se agrega una ronda de Repechaje donde los equipos "extra" juegan entre sí; los restantes tienen BYE (avanzan directamente).
- Las rondas posteriores se crean con `placeholder_local` / `placeholder_visitante` (ej: `"Ganador SF1"`) sin equipos asignados.
- Cuando se carga la planilla de un partido y queda en estado `TERMINADO`, el backend identifica automáticamente al ganador y reemplaza el placeholder correspondiente en la siguiente ronda (`avanzar_ganador`).

Nombres de ronda según cantidad de partidos: `Repechaje`, `Octavos de Final`, `Cuartos de Final`, `Semifinal`, `Final`.

Planteles (/api/planteles)

Método	Ruta	Rol mínimo
GET	/activo/{id_equipo}	Público
GET	/id/integrantes	Público
POST	/	Admin
POST	/integrantes	Admin
PATCH	/id/cerrar	Editor
DELETE	/id	Admin
DELETE	/integrantes/id	Admin

Fichajes (/api/fichajes)

Método	Ruta	Rol mínimo
GET	/club/{id}	Público
GET	/disponibles	Editor
POST	/	Admin
PATCH	/id/baja	Admin

Noticias (/api/noticias)

Método	Ruta	Rol mínimo
GET	/	Público
GET	/id	Público
POST	/	Admin
PUT	/id	Admin
DELETE	/id	Admin
POST	/preview-url	Admin

Estadísticas y vistas

Método	Ruta	Descripción
GET	<code>/api/stats/global</code>	Estadísticas globales
GET	<code>/api/vistas/torneos/{id}/posiciones</code>	Tabla de posiciones
GET	<code>/api/vistas/goleadores/{id_torneo}</code>	Goleadores del torneo
GET	<code>/api/vistas/valla-menos-vencida/{id_torneo}</code>	Valla menos vencida
GET	<code>/api/vistas/tarjetas-acumuladas</code>	Ranking de tarjetas

Manejo de errores

Todos los errores siguen el formato:

```
{
  "code": "CODIGO_ERROR",
  "message": "Descripción en español"
}
```

Códigos de estado usados: `400`, `401`, `403`, `404`, `409`, `422`, `429`, `500`.

Frontend — React

Estructura de `src/`

```
src/
├── api/                # Llamadas a la API agrupadas por entidad
│   ├── config/        # Instancia Axios con baseURL e interceptores
│   └── vistas/         # Endpoints de vistas especiales
├── pages/
│   ├── public/        # Páginas sin autenticación
│   └── admin/         # Páginas del panel admin (requieren auth)
└── components/        # Componentes reutilizables
```

Convenciones

- Componentes: `PascalCase`
- Variables y funciones: `camelCase`
- Llamadas a la API: funciones en `src/api/<entidad>.api.ts`
- Rutas protegidas: usan un componente `ProtectedRoute` que verifica el token y el rol

Rutas principales

Ruta	Tipo	Descripción
/	Público	Home
/public/clubes	Público	Listado de clubes
/public/posiciones	Público	Tabla de posiciones
/public/ranking	Público	Goleadores y estadísticas
/fixture	Público	Próximos partidos
/resultados	Público	Partidos jugados
/noticias	Público	Noticias
/login	Público	Login
/admin	Auth	Dashboard admin
/admin/partidos	Editor+	Gestión de planillas
/admin/fixture	Editor+	Gestión de fixture
/admin/noticias	Admin+	Gestión de noticias
/admin/personas	Admin+	Gestión de personas
/admin/equipos/:id	Admin+	Gestión de equipos
/admin/clubes	Superusuario	Gestión de clubes
/admin/torneos	Superusuario	Gestión de torneos
/login/usuarios	Superusuario	Gestión de usuarios

Exportación de fixture a PDF

Módulo: [frontend/src/pages/admin/fixture/exportarFixturePDF.ts](#)

Dependencias: [jsPDF](#) + [jspdf-autotable](#) (ya listadas en el stack).

API pública:

```
exportarFixturePDF(partidos: FixturePartido[], torneo: Torneo, esPlayoff: boolean): void
```

Genera y descarga el PDF directamente desde el navegador. No requiere llamada al backend.

Estructura del documento (A4 portrait):

- Encabezado con título, nombre del torneo ([nombre](#) · [categoría](#) · [división](#) · [género](#)) y fecha de exportación.
- Línea separadora en azul institucional.
- Cuerpo en dos columnas, cada una con tablas de [jspdf-autotable](#) (grid theme, filas alternadas).

Layout según tipo de torneo:

Tipo	Función interna	Agrupación
Liga	<code>exportarLiga</code>	Por <code>numero_fecha</code> ; muestra rueda (ida/vuelta) y equipo que descansa si aplica
Playoff / Copa	<code>exportarPlayoff</code>	Por <code>nombre_ronda_playoff</code>

Salto de página: se estima la altura de cada bloque antes de renderizarlo. Si no entra en la página actual, se agrega una nueva página y se reinician ambas columnas.

Nombre del archivo: `fixture_<nombre_torneo_en_snake_case>.pdf`

Build para producción

```
docker compose up --build
```

El frontend se construye con `vite build` dentro del contenedor y se sirve desde Nginx.

Autenticación y seguridad

Flujo JWT

1. El usuario hace POST `/api/auth/login` con credenciales.
2. El servidor responde con:
3. **Access token** (JWT de corta duración, en el body)
4. **Refresh token** (en cookie HTTP-only, 30 días)
5. El frontend incluye el access token en el header `Authorization: Bearer <token>`.
6. Al vencer el access token, el frontend llama a `/api/auth/refresh` (la cookie se envía automáticamente).
7. El servidor valida el refresh token y emite nuevos tokens.

Roles y jerarquía

```
SUPERUSUARIO > ADMIN > EDITOR > LECTOR
```

Las dependencias FastAPI `require_superuser`, `require_admin`, `require_editor` en `dependencies/permissions.py` se usan como parámetros en cada endpoint.

Seguridad adicional

- **Rate limiting:** login limitado a 5 req/min por IP (SlowAPI).

- **Hashing de contraseñas:** argon2 (con fallback a bcrypt vía passlib).
- **Cookies:** `HttpOnly`, `Secure=True` en producción, `SameSite=strict`.
- **CORS:** orígenes permitidos configurados desde `.env`.
- **Invitación de usuarios:** los nuevos usuarios se registran solo desde un enlace de email con token temporal; no hay registro público.

Migraciones

El proyecto usa **Alembic** para migraciones de esquema. La inicialización base se hace por SQL (scripts en `db/init/`). Alembic se usa para cambios incrementales después del deploy inicial.

Consultar `ALEMBIC_MANUAL.txt` en la raíz para el flujo de trabajo detallado.

Historial de migraciones

Versión	Descripción
<code>0001</code> – <code>0012</code>	Migraciones iniciales: tablas base, auditoría, fixture básico
<code>0013</code>	Mejoras de fixture: agrega <code>PENDIENTE</code> al enum <code>tipo_estado_partido</code> ; columna <code>actualizado_por</code> en <code>fixture_partido</code> ; columnas <code>actualizado_en</code> , <code>actualizado_por</code> e <code>id_equipo_descansa</code> en <code>fixture_fecha</code> ; FK <code>fixture_fecha → equipo</code> para el equipo que descansa
<code>0014</code>	Tipo de torneo: crea enum <code>tipo_torneo</code> (<code>LIGA</code> , <code>PLAYOFF</code> , <code>COPA</code>) y agrega columna <code>tipo</code> en tabla <code>torneo</code> (default <code>LIGA</code>)
<code>0015</code>	Sistema de playoff: crea tabla <code>fixture_playoff_ronda</code> ; agrega en <code>fixture_partido</code> las columnas <code>id_fixture_playoff_ronda</code> (FK), <code>placeholder_local</code> , <code>placeholder_visitante</code> ; hace nullable <code>id_equipo_local</code> e <code>id_equipo_visitante</code> ; actualiza constraint de equipos distintos para permitir NULLs

Aplicar migraciones en producción (VPS)

```
# 1. Subir los cambios al servidor (git pull o scp)
# 2. Reconstruir e iniciar los contenedores
docker compose up --build -d

# 3. Aplicar migraciones pendientes
docker compose exec api alembic upgrade head

# 4. Verificar
docker compose exec api alembic current
```

Comandos básicos:

```
# Generar nueva migración (dentro del contenedor api)
docker compose exec api alembic revision --autogenerate -m "descripcion"

# Aplicar migraciones pendientes
docker compose exec api alembic upgrade head

# Ver historial
docker compose exec api alembic history
```

Backups

Los scripts en `backups/` gestionan el backup y restauración de la base de datos. Generan archivos `.dump` (formato binario de PostgreSQL, más eficiente que SQL plano).

Scripts disponibles

Script	Uso
<code>backup_hockey.sh</code>	Genera un backup y opcionalmente lo sube a Google Drive
<code>restore_hockey.sh</code>	Restaura la base desde un archivo <code>.dump</code>

Ambos scripts reciben un argumento de entorno (`local` o `vps`) y leen su configuración desde un archivo `backups/.env.<entorno>`.

Archivo de configuración por entorno

Crear `backups/.env.local` y/o `backups/.env.vps` con:

```
DB_NAME=hockey
DB_USER=hockey_user
CONTAINER_NAME=hockey_db
BACKUP_PATH=/ruta/donde/guardar/los/dumps
```

Hacer un backup manualmente

```
# En entorno local
./backups/backup_hockey.sh local

# En el VPS
./backups/backup_hockey.sh vps
```

El archivo generado tiene el formato: `backup_hockey_YYYY-MM-DD_HH-MM-SS.dump`

En el entorno `vps`, además de guardar el archivo localmente:

- Lo sube a **Google Drive** usando `rclone` (carpeta `backups/hockey`)
- Elimina de Drive los backups con más de 30 días
- Elimina del disco local los backups con más de 30 días

Restaurar desde un backup

```
# En entorno local
./backups/restore_hockey.sh local /ruta/al/archivo.dump

# En el VPS
./backups/restore_hockey.sh vps /ruta/al/archivo.dump
```

Atención: la restauración usa `--clean --if-exists`, lo que elimina y recrea las tablas antes de restaurar. No ejecutar en producción sin confirmar primero.

Backup automático en el VPS

El backup automático está configurado como un cron job en el VPS que ejecuta el script a la **1:00 AM**. Para verificarlo o editarlo:

```
crontab -l      # Ver cron jobs activos
crontab -e      # Editar
```

La línea configurada en el VPS es:

```
0 1 * * * /root/proyectos/pagina-hockey/backups/backup_hockey.sh vps >> /root/proyectos/
backups/backup.log 2>&1
```

El log se guarda en `/root/proyectos/backups/backup.log`.

Dependencia: rclone

El script de VPS requiere `rclone` instalado y configurado con acceso a Google Drive. Para verificar:

```
rclone listremotes      # Debe aparecer "drive:"
rclone ls drive:backups/hockey # Lista los backups en Drive
```

Si `rclone` no está configurado, seguir la [guía oficial de rclone con Google Drive](#).

Deploy en producción

Requisitos del servidor

- Docker y Docker Compose
- Puerto 80 disponible
- Dominio apuntando al servidor (`tu-dominio.com.ar`)

Pasos

1. Clonar el repositorio en el servidor.

2. Crear el archivo `.env` con valores de producción (ver sección [Configuración del entorno](#)).
3. Asegurarse de que `COOKIE_SECURE=True`, `ENVIRONMENT=production`, `ENABLE_SWAGGER=False`.
4. Levantar:

```
bash docker compose up --build -d
```
5. Verificar que los contenedores corren:

```
bash docker compose ps
```

Nginx

El archivo `nginx.conf` configura el proxy para producción con el dominio `tu-dominio.com.ar`. Para desarrollo local se usa `nginx.local.conf`.

Variables de entorno críticas para producción

Variable	Valor recomendado
<code>COOKIE_SECURE</code>	<code>True</code>
<code>ENVIRONMENT</code>	<code>production</code>
<code>ENABLE_SWAGGER</code>	<code>False</code>
<code>JWT_SECRET</code>	String aleatorio de al menos 64 caracteres
<code>POSTGRES_PASSWORD</code>	Contraseña fuerte
<code>FRONTEND_URL</code>	URL pública del sitio
<code>CORS_ALLOW_ORIGINS</code>	URL pública del sitio

Logs y diagnóstico

```
# Ver logs en tiempo real de todos los servicios
docker compose logs -f

# Logs de un servicio específico
docker compose logs -f api
docker compose logs -f frontend
docker compose logs -f db

# Abrir shell en el contenedor de la API
docker compose exec api bash

# Conectarse a la base de datos
docker compose exec db psql -U hockey_user -d hockey
```